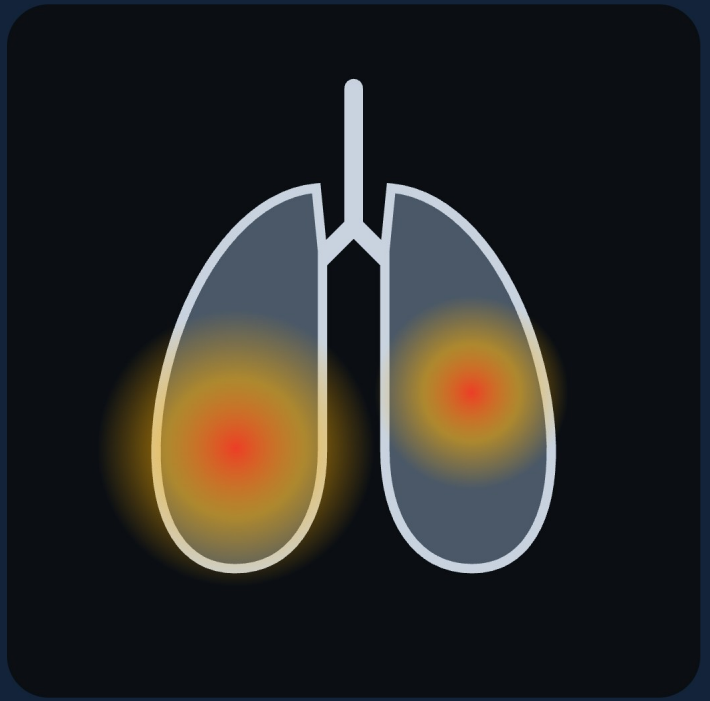
4조 · 6일차 AI 폐렴 예측

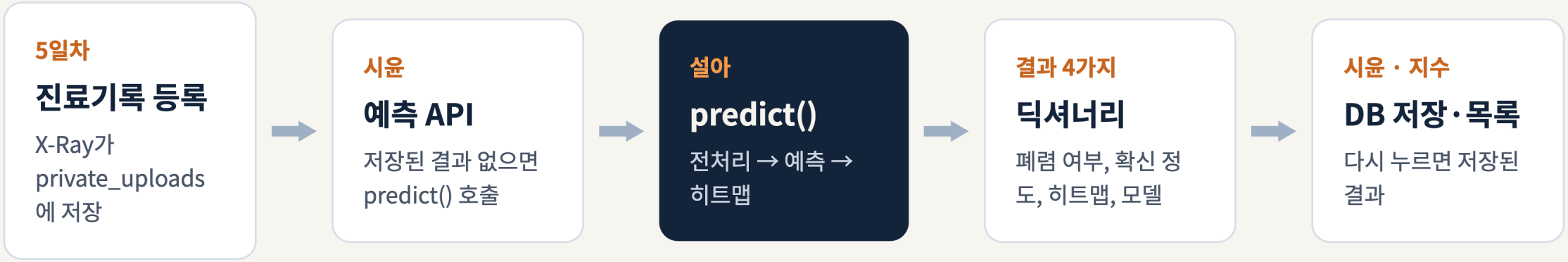
모델 코드 설명

worker/model.py · worker/check_model.py 설명



predict()가 쓰이는 곳

전체 흐름



오늘 설명할 부분은 가운데 **predict()** 하나예요. 나머지 코드는 이 함수가 돌려주는 4가지만 알면 돼요.

해커톤 때 문제

폐만 잘라서 보는 이유

0.99

검증 점수 (학습과 비슷한 사진)

0.85

실제 테스트 점수 (검은 띠 없는 사진)

0.95

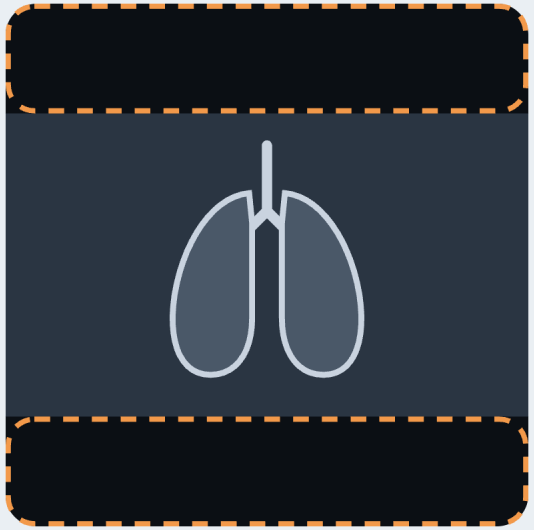
띠 제거 + 폐만 자른
우리 모델(E8) 테스트 점수

폐렴 사진은 원래 가로로 길어서 검은 띠가 정상의 약 2배(15% → 31%) -> 이 패턴을 모델이 외워버림
"띠가 두꺼우면 폐렴"으로 외워버린 거라, 띠와 폐 바깥을 지우고 폐 안만 보게 했습니다.

출처: DACON 해커톤 최종 보고서 (E4 → E8)

model.py

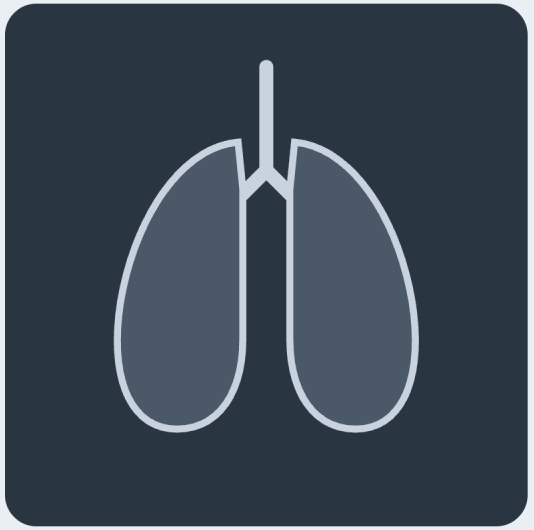
전처리 1~4단계



1. 224 맞추기

흑백, 비율 유지, 남는 곳은 검은색

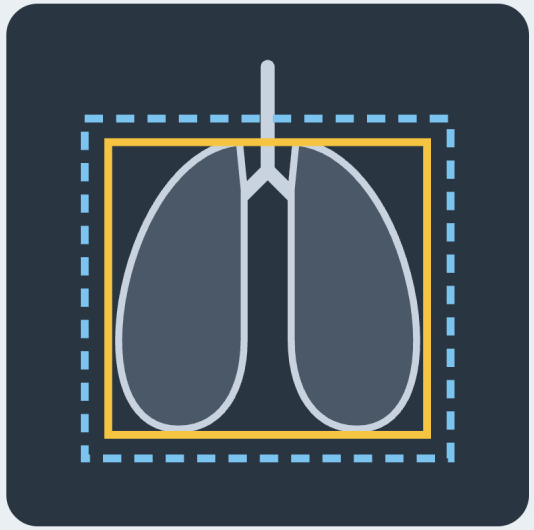
`preprocess`



2. 검은 띠 제거

밝기 20 이하인 줄을 잘라냄

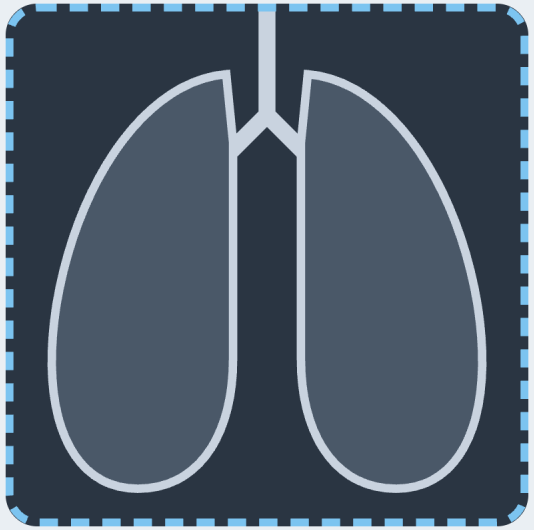
`remove_band`



3. 폐 위치 찾기

노랑: 폐 상자 · 하늘: 8% 여백

`find_lung_box`



4. 폐만 자르기

하늘색 상자만 남겨 224로

`crop_lung`

단계를 단순하게 나타낸 그림 (실제 X-Ray 사진은 데이터 이용 규칙 때문에 넣지 않음) · 폐를 못 찾으면(5% 미만) 사진 전체 사용

model.py 맨 위 · 학습 때와 같아야 함

설정값

이름	값	뜻
IMG_SIZE	224	모델에 넣는 사진 크기 (가로·세로)
BAND_THRESHOLD	20	이 밝기 이하인 줄은 검은 띠로 보고 잘라냄
SEG_SIZE	512	폐 위치 찾는 모델(PSPNet)에 넣는 크기
LUNG_MIN_RATIO	0.05	폐 픽셀이 5% 미만이면 못 찾은 것 → 사진 전체 사용
LUNG_MARGIN	0.08	폐 상자 바깥으로 넓히는 여백 8%
THRESHOLD	0.5	폐렴 확률이 이 값 이상이면 폐렴으로 판정

5단계 입력 변환(TRANSFORM)도 그대로예요: 흑백 1장을 3장으로 복제 → ImageNet 평균·표준편차로 정규화

해커톤 노트북 Final_submission.ipynb 3·4·5번 셀과 같은 값

model.py 5단계

예측과 confidence

ResNet18
점수 2개 (0번 정상, 1번 폐렴)

→

softmax
합이 1인 확률로,
1번이 폐렴 확률 p

→

p ≥ 0.5 ?
맞으면 폐렴, 아니면 정상

→

confidence
판정한 쪽의 확률 × 100

폐렴 사진 · p = 0.9973

폐렴 → 99.73

p 그대로

정상 사진 · p = 0.0046

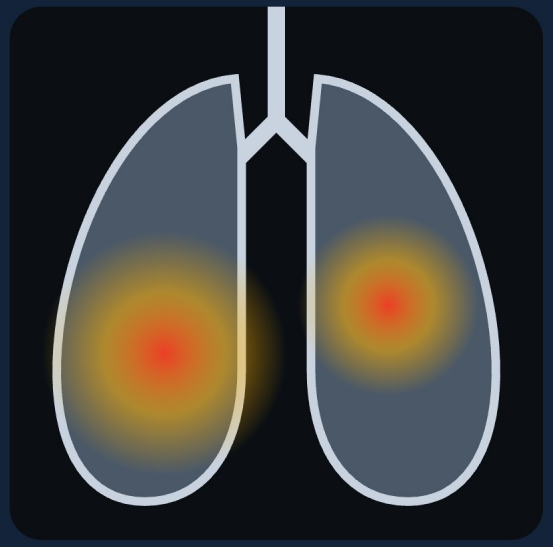
정상 → 99.54

1 − p = 0.9954

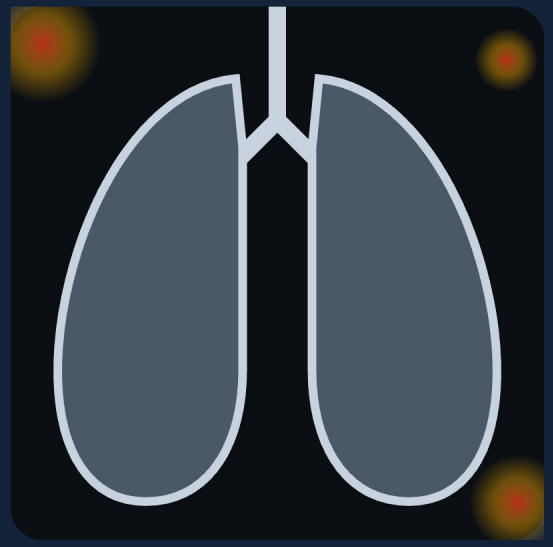
판정 기준이 0.5라서 confidence는 항상 50 이상 · 소수점 둘째 자리까지 저장

model.py 6단계

Grad-CAM 히트맵



폐렴 · 폐 안쪽이 빨강



정상 · 모서리에 잡음

1. ResNet18 마지막 층(layer4)의 특징 맵을 저장
2. 폐렴 점수에서 거꾸로 계산해 특징마다 중요도 구하기
3. 중요도만큼 더해 7×7 지도 → 224로 키우기
4. 폐 사진 위에 색으로 겹쳐 PNG 저장

히트맵 모양을 단순하게 나타낸 예시 그림 · 정상으로 확신한 사진은 폐렴 근거가 없어서 잡음처럼 보임 · 히트맵을 못 만들어도 예측 결과는 그대로 돌려줌

노트북 코드와 달라진 점

서버용으로 바꾼 점

1. 서버 켤 때 한 번만 로딩

main.py lifespan → load_models() → _models 보관함에 저장.
요청마다 꺼내 쓰기만 함.

2. 안전한 로딩

weights_only=True: 가중치 숫자만 읽고 파일 속 코드는 실행 안 함.
파일이 없으면 "드라이브에서 받아주세요" 오류.

3. 한 번에 하나씩 예측

_lock: 히트맵 계산 중간값이 동시 요청끼리 섞이지 않게 잠금.
한 장에 0.6초라 그렇게 오래 기다리지는 않음.

4. 히트맵 실패해도 결과는 반환

try/except로 감싸서 실패하면 로그만 남기고 heatmap_path = None. 그래서 heatmap_url이 nullable

시윤님: predict()는 오래 걸리는 일반 함수라 async 라우터에서는 run_in_threadpool로 감싸서 호출

worker/check_model.py

서버 없이 모델 확인

```
uv run python -m worker.check_model 사진폴더/*
```

사진마다 판정, confidence, 걸린 시간을 출력하고 히트맵은 worker/check_outputs/에 저장 (커밋 안 됨)

0.60초
이후 예측 평균

0.80초
첫 예측 (3초 기준 통과)

사진	노트북	서버
폐렴 1	99.73	99.73
폐렴 2 (JPEG)	100.00	100.00
폐렴 3	99.99	99.99
정상 1 (JPEG)	99.54	99.54
정상 2	98.92	98.92
정상 3	99.92	99.92

검증 데이터 6장 · 6장 모두 판정 정답, confidence 동일 → 전처리가 노트북과 똑같이 옮겨졌다는 뜻 · MacBook Air CPU 기준